SPHERE App — User Handbook

What is SPHERE?

SPHERE (Synthetic Privacy-preserving Honest Evaluation and Release Engine) is the end-to-end platform for generating, certifying, and sharing privacy-preserving synthetic data:

1. **Generate** — produce a synthetic twin of your sensitive dataset
2. **Evaluate & Certify** — formally score fidelity and privacy, issue a certificate
3. **Share** — upload the synthetic data to Dropbox or Zenodo
4. **Post to Catalog** — list the dataset in the SPHERE World public directory
5. **Analyse with AI** — use Claude to develop and run analyses on the synthetic data, then apply the resulting code to your real dataset locally

One rule governs all five steps: **your real data never leaves your machine.**

Other things do leave, and you should know which. You choose when to upload a synthetic file (step 3) and when to post catalog metadata (step 4). Whenever you use the AI features (step 5), your conversation is routed through SPHERE's own servers and stored there — whether you are spending SPHERE credits or using your own Anthropic key. And after each generate, evaluate, certify or share, the app sends SPHERE a small usage record — which step, row and column counts, file size, duration — tied to your account. It never contains your data, file names or column names. Generation, evaluation and certification never need a network: once you are signed in they work offline, and the record waits on your Mac until a connection is available. All of this is documented in *What SPHERE's servers receive* below.

Privacy Architecture

Understanding the privacy boundaries is central to using SPHERE confidently. This section covers all five panels at once so you know exactly what stays local and what leaves.

What Stays on Your Computer

Item	Why it stays local
Your original (real) data	Read only by SPHERE's own code on this Mac — to synthesise the twin, hash it and score it. Never uploaded, never transmitted, never sent to the model.
Synthesis algorithm	Runs entirely in the app process on your machine.
Evaluation computation	All statistical comparisons (fidelity, privacy tests) run locally.
SPHERE certificate	Generated locally, saved as an HTML file on your disk.
Dropbox and Zenodo tokens	Stored encrypted via macOS Keychain. Used only to talk to those services directly. Never sent to SPHERE's servers.
Your Anthropic API key	Stored encrypted via macOS Keychain. It is **forwarded through SPHERE's server** on each AI request so usage can be metered — it passes through and is not stored there.
Column names, dimensions & scores used in AI auto-fill	Derived from the **synthetic** file only — not the real one.
analysis.py and all session figures	Written and executed entirely inside an isolated local sandbox.

Generation, evaluation and certification need no network and, once you are signed in, work offline. The only thing any of them sends is the usage record described in the next section — never your data, file names, paths, column names or values.

What SPHERE's Servers Receive

The AI features do not talk to Anthropic directly. Every request is routed through SPHERE's own proxy at `api.sphereworld.ai`, which is how usage is metered against your account. This applies whether you are spending SPHERE credits **or** using your own Anthropic key.

That server receives, and **retains**:

- **The full conversation** — every message you send and every reply the model returns, including tool results and any real-data results you approved sending for the report.
- **Usage metadata** — token counts, model, timestamps, and your account identity.

It does **not** receive your real data, your real file's path, or your Anthropic key in storable form (the key is forwarded upstream on each request and not kept).

Sign-in is required to use the app, so this applies to all AI use. If your work is governed by an agreement that forbids conversation content leaving your institution, use the Generate, Evaluate and Share tabs and not the AI features — those tabs send SPHERE no conversation content and no data, only the usage record described next.

Usage records. After each successful generate, evaluate, certify or share, the app saves a small record on this Mac and sends it to SPHERE, tied to your SPHERE account, whenever a connection is available. It is always on for signed-in users; there is no setting to turn it off. Each record contains only:

- **The step** — generate, evaluate, certify or share — and, for a share, its destination (Dropbox, Zenodo, an S3 bucket or the SPHERE World catalog)
- **Row count, column count and file size**
- **Duration** of the step, and the time it completed
- **App version**

It never contains your data, file names, paths, column names, cell values or dataset titles. Your account is identified by your sign-in, not by anything in the record.

Generate, evaluate and certify never need a network and work offline once you are signed in. Signing in itself needs a connection: the first time, and again after 7 days without one. A record made offline waits on your Mac and is sent the next time the app is running with a connection: after your next step, at the next launch, or within 15 minutes. Sending happens in the background and never delays, blocks or fails the step itself.

What Is Uploaded (Synthetic Data Only)

When you share a dataset, **only the synthetic CSV** is uploaded — never the real data.

- **Dropbox**: Synthetic CSV and certificate go to your Dropbox account via OAuth. SPHERE has no access beyond the folder it creates.
- **Zenodo**: Synthetic CSV and certificate go to your Zenodo account via your personal API token.

What the SPHERE World Catalog Receives

When posting to the public catalog, the API receives metadata only — no data rows, ever:

- Dataset title, description, domain, sharing terms
- Public Dropbox/Zenodo link to the *synthetic* file
- Fidelity and privacy scores (aggregate numbers only)
- Whether the dataset was SPHERE-generated or externally produced
- Whether a certificate exists · Storage method (Dropbox or Zenodo)

What Claude Receives — Auto-Fill (Catalog Post)

The **Generate with AI** button in the Post panel sends a prompt to Claude (Anthropic API) containing:

1. The synthetic CSV **filename** (not its contents)
2. Column **names** from the synthetic header (up to 10, then "etc.")
3. **Dimensions** — row and column counts of the synthetic file
4. **Aggregate scores** — fidelity and privacy composites
5. Whether the dataset was SPHERE-generated or external

What Claude never receives: individual data records, cell values, your real data, or any information that could identify research participants. Note that column *names* are sent (item 2 above); in some domains a column name is itself sensitive.

What Claude Receives — SPHERE AI Panel

In the SPHERE AI tab, Claude operates an agent loop over a Python sandbox. It receives:

- The **contents of the synthetic CSV** (Claude reads it via Python tools, e.g. `pd.read_csv`)
- Conversation messages and tool results (stdout/stderr from local Python execution)
- Figures it requests to view
- Files you explicitly attach in the chat

What Claude receives, and what it does not

Claude never receives your real dataset. The file is never uploaded, and neither is its path. Every tool the agent can call — `read_file`, `list_files`, `write_file`, `view_figure` — resolves only inside that session's own directory, and during a deploy the real CSV is staged into a *separate* directory that the sandbox refuses to open or even list.

Individual records are not sent. The analysis is instructed, as a hard rule, to print aggregates only — counts, means, model coefficients, group sizes — and to describe data-quality problems by **column name, row position and count** rather than by showing the rows. If it needs to show what a problem row looks like, it uses the SPHERE twin, which is what the twin is for.

What is sent, and when:

Channel	When	What
Error traceback	Automatically, if `analysis.py` fails on real data — up to 3 attempts	File name, line number and exception class only. The exception **message is dropped entirely**, and source lines are never echoed.
Analysis output + figures	Only when you press **Send this and write the report**	The exact text shown to you on screen, plus the figures as rendered images.

Nothing else from a real run reaches Claude.

Before anything is sent, you see it

Pressing **Review what will be sent** does not send anything. It shows you the exact characters that would leave the machine, plus thumbnails of every figure. Only then does **Send this and write the report** transmit — and it transmits precisely the string you were shown, not a recomputed one.

Two filters run before that text is displayed, and both are described honestly:

- **Identifier scrub.** SPHERE reads your real CSV in its own process, collects values from identifier-like columns (MRNs, names, accession numbers), and removes any line containing one. Those values never enter the interface process. If your file has no such column — which is normal for de-identified extracts — the app tells you so and removes nothing.
- **Size cap.** Output is capped, and if the cap is reached the notice appears *inside* the text you review, so you can see that something was cut.

Sandbox enforcement

Code the agent writes never runs inside SPHERE. It runs as a separate frozen binary launched through `/usr/bin/sandbox-exec` in a macOS Seatbelt profile it cannot renegotiate. There is no unconfined fallback: if the bundled runtime is missing, the tool call fails rather than running unprotected. Inside that profile it:

- has **no network** — TCP, UDP, DNS and unix sockets are all denied
- has **no credentials** in its environment (an allow-list of eight variables)
- can **write** nowhere but its own session directory
- can **read** exactly one data file, named by literal path — the twin, or during a deploy the staged real copy
- cannot read your stored API key, other sessions' outputs, your home directory, cloud-synced folders, other applications' data, or external volumes

Child processes inherit the sandbox, and symlink, hard-link, firmlink and nested-sandbox escapes were each attempted and failed. `npm run test:boundary` runs these as a test suite against the real binary rather than asserting them.

Limits you should know about

Honest disclosure of what this design does **not** guarantee:

1. **Figures are sent as rendered images.** A per-subject plot shows one point per person. This is deliberate — it is how the report compares twin against real — but it means figures carry individual-level structure. Review the thumbnails.
2. **The no-records rule is an instruction, not a lock.** The agent writes the analysis, so nothing inspects the script before it runs. The identifier scrub is a backstop and cannot recognise a table of per-subject numbers that prints no identifier. The review step is what actually enforces this — please read it.
3. **The sandbox denies by name, not by allow-list.** Python needs parts of `~/Library` to start, so the profile permits by default and denies specific areas. Locations outside both the deny list and your data are readable.
4. **SPHERE's own code does read your real file** — to stage it, hash it, synthesise the twin, and score privacy and fidelity. That is fixed application code the model never influences, and it is not sandboxed. It is, however, launched with a stripped environment: your API key and every other credential in the app's environment are withheld from it, so a bug in that code cannot turn into a network call on your account. "Claude never sees your real data" and "SPHERE never touches your real data" are different statements; only the first is true.

The SPHERE Workflow

All five tabs form a single linear workflow. You do not have to complete every step — use as many as are relevant to your work.

Step 1 — Generate Synthetic Data

Tab: Generate

1. Drag your real CSV into the app, or click to browse. The file is read locally — nothing is uploaded.
2. Configure generation parameters (synthetic rows, noise level, random seed).
3. Click **Generate**. The SPHERE algorithm runs on your machine and produces a synthetic CSV.
4. Download the synthetic CSV to a location of your choice.

Your real data is only ever read from disk — it is never stored by the app or transmitted anywhere.

Step 2 — Evaluate & Certify

Tab: Evaluate, Certify & Share

1. Load your real CSV and the synthetic CSV to evaluate (SPHERE-generated or from any other tool).
2. Click **Evaluate**. The app computes: - **Fidelity scores** — how closely the synthetic data matches the real data's statistics (mean, variance, correlations, KS distance). - **Privacy scores** — how resistant the synthetic data is to singling-out, linkability, and inference attacks.
3. Review the scores and the visual report.
4. Click **Generate Certificate** to create a signed HTML evaluation report, saved to your disk.

All computation happens on your machine. No data leaves during this step.

Step 3 — Share to Cloud

Tab: Evaluate, Certify & Share → Share section

Three options are available:

Option A: Upload to Dropbox Connect via OAuth (direct with Dropbox, not SPHERE). Click **Share to Dropbox** — the synthetic CSV and certificate are uploaded to a new folder and a public link is generated.

Option B: Upload to Zenodo Enter your Zenodo API token (stored locally via Keychain). Click **Upload to Zenodo** — the files are uploaded and a permanent DOI is generated.

Option C: Save as ZIP Download a ZIP of the synthetic data and certificate for manual distribution. Nothing is uploaded.

Step 4 — Post to SPHERE World Catalog

Tab: SPHERE World

Once your synthetic data is hosted on Dropbox or Zenodo, you can make it discoverable in the public catalog at sphere-world.vercel.app.

1. On your dataset card, click 🌐 **Post to SPHERE World**.
2. Fill in the title and description manually, or click **Generate with AI** to auto-fill. Set the domain and sharing terms: - ◎ **Open (with citation)** — anyone can download; just cite your work. - ✉

Request access — interested parties must email you first. - ⊙ **Revenue share** — commercial licensing terms apply.

3. Click **Post**. Only metadata and the public link are sent — no data rows.
4. The button changes to 🌐 **Update on SPHERE World**. Click it any time to change the title, description, or sharing terms. The same catalog record is updated in place — no duplicate is created.

Deduplication: Each dataset has a stable local UUID. Re-posting always updates the same catalog record. If you try to post a dataset whose title already exists in the catalog (posted by you or someone else), you will be asked to choose a different title.

Step 5 — Analyse with SPHERE AI

Tab: SPHERE AI

SPHERE AI lets you build a rigorous data analysis by chatting with Claude — using the synthetic data as a safe stand-in throughout development. When the analysis is ready, the app runs it locally against the real data. **Claude is not involved in that final execution step at all.**

Setup

- **Anthropic API key** — obtained at `console.anthropic.com/settings/keys`. Entered once, stored encrypted via macOS Keychain.
- **Model** — Claude Opus 5 (most capable), Fable 5 (extended thinking), Sonnet 5 (balanced), Haiku 4.5 (fastest). Changed in the Settings panel.

How the session works

1. **Load a synthetic CSV.** Click the green *Synthetic* chip and pick your CSV, or click a built-in example. An isolated sandbox folder is created and the synthetic file is placed inside it.
2. **Load the real CSV (optional, needed for deploy).** Click the red *Real* chip. This path is stored in the app process only — it is never shared with Claude.
3. **Chat with Claude.** Type a prompt and press Enter (⇧Enter for new line). Built-in starter prompts: - "Summarize the dataset: column types, missing values, and key distributions." - "Identify the strongest associations between variables and quantify them with effect sizes and p-values." - "Propose three hypothesis-driven analyses, then run the most informative one."
4. **Claude writes and runs code.** The agent iteratively writes Python in `analysis.py`, installs packages (`matplotlib`, `seaborn`, `statsmodels`…), and executes in the sandbox. Figures appear in the right panel. All execution is local.

5. **Iterate.** Ask follow-ups, change chart styles, add statistical tests. Claude refines `analysis.py` across turns. Click **Review analysis.py** to inspect the current script.
6. **Run on real data.** Once the synthetic run shows a green exit code, click **Deploy on real**. The app (not Claude): - Copies your real CSV into the sandbox as a temporary file. - Runs `analysis.py` locally as a subprocess against that copy. - Immediately deletes the copy when the script finishes. - Shows the real-data figures alongside the synthetic figures.

No message is sent to Claude during this step. Claude already finished its work on the synthetic data.

7. **Save the session report.** After a successful run, a self-contained HTML report is generated containing the full conversation transcript, `analysis.py`, both figure sets, and an AI-written narrative summary. Click **Open report in sandbox** to reveal it in Finder.

File attachments

Click **+** in the chat input bar to attach files (PDFs, images, text/code). Attached files are sent to the Claude API as conversation context. Do not attach files containing real personal data.

Tools Claude can use

Tool	What it does
`python`	Executes Python code in the sandbox. Output and errors are returned to Claude.
`pip_install`	Installs a Python package into the sandbox environment.
`read_file`	Reads a file from the sandbox directory.
`write_file`	Writes or overwrites a file in the sandbox directory.
`list_files`	Lists files present in the sandbox directory.
`view_figure`	Renders a saved figure so Claude can inspect and refine it.

Buttons reference

Button	What it does
Send	Sends your message to Claude. Press Enter (⇧Enter for new line).
■ Stop	Interrupts Claude at the next turn boundary.

Button	What it does
Review analysis.py	Shows the current state of the analysis script in a dialog.
Deploy on real	App runs `analysis.py` locally against your real CSV, then sends the *results* to Claude for the comparison section.
Sandbox folder	Opens the sandbox directory in Finder (scripts, figures, report).
↺ Reset	Clears the conversation and starts a fresh session with the same synthetic CSV.

First-launch pre-loading

The first time the SPHERE AI tab is used, `matplotlib`, `seaborn`, and `statsmodels` are pre-installed in the sandbox Python environment. A progress banner is shown (typically 30–90 seconds). Subsequent sessions start immediately.

Managing Your Datasets

Syncing and Deleting

Delete from Dropbox / Zenodo: Click *Delete from Dropbox* (or Zenodo) on any dataset card. The app deletes the remote folder, removes the local history entry, and automatically removes the catalog listing.

Sync: Click **Sync** in the SPHERE World tab to reconcile your local history with remote storage. If a file was deleted outside the app (e.g. directly in Dropbox's web UI), the sync detects it and removes the stale entry and catalog listing.

Updating a Catalog Listing

Click 🌐 **Update on SPHERE World** on any dataset card to change the title, description, domain, or sharing terms. The existing catalog record is updated — no duplicate is created.

SPHERE vs. External Synthetic

The app and catalog distinguish two dataset origins:

- **SPHERE** badge (cardinal red): The synthetic data was generated by the SPHERE algorithm built into this app.
- **External synthetic** badge (grey): The data was generated by a third-party tool (e.g., SDV, synthpop, CTGAN) and then evaluated using SPHERE's metrics.

This distinction is shown on dataset cards in both the app and the catalog.

Data Flow Summary

```
Your real data (CSV)
        │
        ▼
  [SPHERE App — on your Mac]
        │
        ├─► Synthesis algorithm ──────────────────────► Synthetic CSV (saved locally)
        │                                                        │
        ├─► Evaluation & scoring (local)                         ├─► Dropbox  (your accou
        │       │                                                └─► Zenodo   (your accou
        │       └─► Certificate HTML (local)                               │
        │                                                                  ▼
        ├─► AI auto-fill (catalog post)                            SPHERE World Catalog
        │     → Claude API receives: filename,                     (metadata + public link
        │       column names, dimensions, scores                    no real data, no rows)
        │
        ├─► SPHERE AI panel
        │     ├─► Synthetic CSV → sandbox
        │     │       Claude writes analysis.py via python / pip / write_file tools
        │     │       All execution: local subprocess on this Mac
        │     │       Claude API receives: conversation + synthetic CSV contents only
        │     │
        │     └─► "Deploy on real" — app-only step, NO Claude API call
        │             App copies real CSV → sandbox → runs analysis.py → deletes copy
        │             Real data never transmitted; Claude is not involved in this step
        │
        └─► Usage record after each generate / evaluate / certify / share
              → saved on this Mac, sent to SPHERE when online (tied to your account):
                step, destination, rows, columns, file size, duration, app version
                — no data, file names, paths, column names or values
```

Frequently Asked Questions

Q: Does SPHERE ever see my real data? Your real data is read on this Mac by SPHERE's own code — to build the twin, hash it and score it — and is never transmitted to SPHERE's servers, Anthropic, Dropbox, Zenodo or any other service. Two other things do reach SPHERE's servers, and neither contains your data: your *conversation* with the AI, which is routed through SPHERE and retained there, and the small usage record sent after each generate, evaluate, certify or share. See *What SPHERE's servers receive*.

Q: Does SPHERE record how I use the app? Can I turn that off? Yes. After each successful generate, evaluate, certify or share, the app saves a usage record on this Mac — the step, the share destination, row and column counts, file size, duration and app version — and sends it to SPHERE, tied to your account, whenever a connection is available. It is always on for signed-in users; there is no opt-out. It never contains your data, file names, paths, column names, cell values or dataset titles.

Q: Do generate, evaluate and certify work without an internet connection? Yes, once you are signed in. None of them needs a network, and offline they run exactly as they do online. Signing in is the one exception: it needs a connection the first time, and after 7 days without one the app asks you to reconnect and sign in again. The usage record for each waits on your Mac until a connection is available. Uploading to Dropbox, Zenodo or S3, posting to the catalog and the AI features do need a connection.

Q: Can the synthetic data be traced back to real individuals? SPHERE evaluates exactly this risk using three attack models: singling-out (can you identify a unique individual?), linkability (can you link records across datasets?), and inference (can you predict a sensitive attribute?). The privacy scores reflect how resistant the synthetic data is to each attack. Higher scores mean greater protection.

Q: Does Claude ever access my real data? No. In the SPHERE AI tab, Claude only ever sees the synthetic CSV and the code it writes itself. When you click "Deploy on real", the app — not Claude — copies your real CSV into the sandbox, runs `analysis.py` as a local subprocess, and immediately deletes the copy. No message is sent during the deploy itself. If you then press the report button, the aggregate results you have reviewed on screen are sent — that is a separate, explicit step.

Q: Is it safe to use SPHERE AI with sensitive datasets? While developing the analysis, Claude works only from the SPHERE twin, which by design contains no real individuals' records — so aim for good privacy scores before you rely on it. On a "Deploy on real" the script runs locally against your real file and Claude receives only the aggregate results, and only after you have read them on screen and pressed send. Your real dataset is never uploaded. See *Limits you should know about* above for what this does not guarantee.

Q: Can Claude access files outside the sandbox? No. Code the agent writes runs in a macOS Seatbelt sandbox with no network and no credentials, able to read exactly one data file — the twin, or during a deploy the staged real copy — and able to write only inside its own session directory. It cannot reach your home directory, cloud-synced folders (iCloud, OneDrive, Box, Dropbox), other applications' data, other SPHERE sessions, your stored API key, or external volumes. Child processes inherit the confinement. One honest caveat: the profile denies by name rather than permitting by allow-list, because Python needs parts of `~/Library` to start — so a location that is neither your data nor on the deny list is readable. `npm run test:boundary` attempts each of these escapes against the real binary.

Q: What exactly is sent to Anthropic's API? It does not go to Anthropic directly — it is routed through SPHERE's proxy, which meters and retains it (see *What SPHERE's servers receive*). The content is: conversation messages, tool results (stdout/stderr from local Python runs), figures Claude requests to view, and files you explicitly attach. The synthetic CSV's contents can be read by Claude via `pd.read_csv` inside the `python` tool. Do not use synthetic data with residual real values if you have concerns about Claude reading column contents.

In the catalog post panel (AI auto-fill): only the filename, column names, dimensions, and aggregate scores from the synthetic file.

Q: Who can access the data on Dropbox or Zenodo? That depends on the sharing terms you set in the catalog. "Open (with citation)" means anyone with the link can access the synthetic file. "Request access" means interested parties must email you first.

Q: What happens if I delete the dataset from Dropbox but it's in the catalog? The next time you run **Sync** in the app, the stale entry is detected and the catalog listing is removed automatically.

Q: Can I update my catalog listing after posting? Yes. Click 🌐 **Update on SPHERE World** on your dataset card at any time. The existing catalog entry is updated in place — no duplicate is created.

Q: Are my API keys and tokens stored securely? All keys are stored in your macOS user data directory, encrypted via macOS Keychain where supported. Your Dropbox and Zenodo tokens are never sent to SPHERE's servers. Your Anthropic key is different: it is forwarded through SPHERE's proxy on each AI request so usage can be metered. It passes through and is not stored there.

Q: Can I evaluate a synthetic dataset generated by another tool? Yes. Use the Evaluate tab with any real CSV and any synthetic CSV, regardless of how the synthetic data was produced. It will be labelled "External synthetic" in the catalog.

Quick Reference

Action	Where	Privacy impact
Generate synthetic data	Generate tab	Local; works offline. Usage record (no data) sent to SPHERE when online
Evaluate fidelity / privacy	Evaluate tab	Local; works offline. Usage record (no data) sent to SPHERE when online
Generate certificate	Evaluate tab	Local; works offline. Usage record (no data) sent to SPHERE when online
Upload to Dropbox / Zenodo	Share section	Synthetic data only; usage record (no data) sent to SPHERE
Post to SPHERE World catalog	SPHERE World tab → Post	Metadata + public link only; usage record sent to SPHERE
AI auto-fill description	Post panel → Generate with AI	Filename, column names, dimensions, scores sent to Claude
Update catalog listing	SPHERE World tab → Update	Same as Post — metadata only
Delete dataset	Card → Delete button	Removes from cloud storage + catalog
Sync	SPHERE World tab → Sync	Checks remote storage, removes stale entries
SPHERE AI — chat session	SPHERE AI tab	Synthetic CSV + conversation, via SPHERE's proxy — conversation retained
SPHERE AI — Deploy on real	SPHERE AI → Deploy on real	Analysis runs locally; results are sent only if you press the report button and approve them
Session report	SPHERE AI → Open report	100% local — HTML saved to sandbox folder